

Topik Pertemuan 1: Pengenalan Microservices

Tujuan Pembelajaran:

1. Memahami konsep dasar arsitektur microservices.
2. Mengetahui perbedaan antara arsitektur monolitik dan microservices.
3. Mengidentifikasi kelebihan dan tantangan dalam mengadopsi microservices.

Materi Pembelajaran:

1. Konsep Dasar Microservices

- **Definisi:**
Microservices adalah gaya arsitektur perangkat lunak di mana aplikasi dibangun sebagai kumpulan layanan kecil yang independen, terpisah, dan dapat di-deploy secara mandiri.
- **Ciri-ciri Microservices:**
 - Setiap layanan memiliki basis kode sendiri.
 - Setiap layanan dapat dikembangkan, di-deploy, dan di-scale secara independen.
 - Layanan berkomunikasi melalui API (biasanya REST atau gRPC).
- **Contoh Aplikasi:**
Aplikasi e-commerce dapat dibagi menjadi layanan terpisah seperti layanan produk, layanan pembayaran, layanan pengiriman, dan layanan pengguna.

2. Perbandingan Arsitektur Monolitik vs Microservices

- **Arsitektur Monolitik:**
 - Seluruh aplikasi dibangun sebagai satu unit tunggal.
 - Semua komponen (UI, logika bisnis, database) terintegrasi dalam satu basis kode.
 - Contoh: Aplikasi tradisional yang di-deploy sebagai satu file WAR/JAR.
- **Arsitektur Microservices:**
 - Aplikasi dibagi menjadi beberapa layanan kecil yang independen.
 - Setiap layanan memiliki basis kode dan database sendiri.
 - Contoh: Aplikasi modern seperti Netflix, Amazon, atau Uber.

- **Perbandingan:**

Aspek	Monolitik	Microservices
Kompleksitas	Mudah di awal, semakin kompleks seiring waktu.	Lebih kompleks sejak awal, tetapi lebih terkelola.
Skalabilitas	Skala seluruh aplikasi.	Skala per layanan sesuai kebutuhan.

Aspek	Monolitik	Microservices
Kecepatan Pengembangan	Lambat jika tim besar.	Cepat karena tim kecil dan independen.
Deployment	Harus di-deploy sebagai satu unit.	Setiap layanan dapat di-deploy mandiri.
Ketahanan	Kegagalan di satu bagian memengaruhi seluruh sistem.	Kegagalan di satu layanan tidak memengaruhi layanan lain.

3. Kelebihan dan Tantangan Microservices

- **Kelebihan:**
 1. **Fleksibilitas:** Setiap layanan dapat dikembangkan menggunakan teknologi yang berbeda.
 2. **Skalabilitas:** Layanan dapat di-scale secara independen berdasarkan kebutuhan.
 3. **Kecepatan Pengembangan:** Tim kecil dapat bekerja secara paralel pada layanan yang berbeda.
 4. **Ketahanan:** Kegagalan di satu layanan tidak memengaruhi layanan lain.
- **Tantangan:**
 1. **Kompleksitas:** Membutuhkan manajemen yang baik untuk komunikasi antar-layanan.
 2. **Operasional:** Membutuhkan infrastruktur yang lebih canggih (Docker, Kubernetes, dll.).
 3. **Konsistensi Data:** Sulit menjaga konsistensi data karena setiap layanan memiliki database sendiri.
 4. **Testing:** Lebih sulit karena melibatkan banyak komponen yang terpisah.

Aktivitas Pembelajaran:

1. **Diskusi Kelas:**
 - Diskusikan studi kasus nyata tentang perbandingan sistem monolitik dan microservices (misalnya: Netflix, Amazon, atau perusahaan lokal).
 - Identifikasi kelebihan dan tantangan yang dihadapi oleh perusahaan tersebut.
2. **Studi Kasus:**
 - Berikan contoh sederhana aplikasi monolitik (misalnya: aplikasi toko online) dan ajak mahasiswa memikirkan bagaimana aplikasi tersebut dapat dipecah menjadi microservices.
3. **Tugas Kecil:**

- Minta mahasiswa mencari contoh perusahaan atau aplikasi yang menggunakan microservices dan menjelaskan alasan mereka mengadopsi arsitektur tersebut.
-

Referensi:

1. Buku:

- "Building Microservices" by Sam Newman.
- "Microservices Patterns" by Chris Richardson.

2. Artikel:

- "What are Microservices?" oleh Martin Fowler.
- "Monolithic vs Microservices Architecture" di DZone.

Topik 2: Prinsip dan Karakteristik Microservices

Tujuan Pembelajaran:

1. Memahami prinsip-prinsip dasar yang mendasari arsitektur microservices.
 2. Mengidentifikasi karakteristik utama dari microservices.
 3. Mampu menganalisis bagaimana prinsip dan karakteristik ini diterapkan dalam sistem nyata.
-

Materi Pembelajaran:

1. Prinsip-Prinsip Microservices

Microservices didasarkan pada beberapa prinsip utama yang membedakannya dari arsitektur monolitik. Prinsip-prinsip ini mencakup:

- **Decoupling (Pemisahan):**
Setiap layanan microservice bersifat independen dan terpisah dari layanan lainnya. Perubahan pada satu layanan tidak memengaruhi layanan lain.
- **Modularitas:**
Aplikasi dibagi menjadi modul-modul kecil (layanan) yang memiliki tanggung jawab tunggal (single responsibility).
- **Independensi:**
Setiap layanan dapat dikembangkan, di-deploy, dan di-scale secara independen.
- **Bounded Context:**
Setiap layanan memiliki batasan domain yang jelas dan bertanggung jawab atas fungsionalitas tertentu.
- **Autonomy (Otonomi):**
Setiap layanan memiliki kendali penuh atas data dan logika bisnisnya.

2. Karakteristik Microservices

Microservices memiliki beberapa karakteristik utama yang membuatnya unik dan efektif:

- **Scalability (Skalabilitas):**
Setiap layanan dapat di-scale secara independen berdasarkan kebutuhan. Misalnya, layanan yang sering diakses dapat di-scale secara horizontal tanpa memengaruhi layanan lain.
- **Resilience (Ketahanan):**
Kegagalan di satu layanan tidak menyebabkan kegagalan sistem secara keseluruhan. Pola seperti circuit breaker dan retry mechanism digunakan untuk meningkatkan ketahanan.
- **Deployability (Kemudahan Deployment):**
Setiap layanan dapat di-deploy secara mandiri tanpa memengaruhi layanan lain. Ini memungkinkan pembaruan dan rilis yang lebih cepat.
- **Technology Heterogeneity (Keberagaman Teknologi):**
Setiap layanan dapat menggunakan teknologi yang berbeda (bahasa pemrograman, database, dll.) sesuai kebutuhan.

- **API-Driven:**
Komunikasi antar layanan dilakukan melalui API (biasanya REST atau gRPC), yang memungkinkan interoperabilitas dan kemudahan integrasi.

3. Contoh Penerapan Prinsip dan Karakteristik

- **Contoh Skalabilitas:**
Pada aplikasi e-commerce, layanan "pencarian produk" dapat di-scale secara terpisah saat terjadi lonjakan traffic (misalnya saat event diskon besar).
 - **Contoh Resilience:**
Jika layanan "pembayaran" mengalami kegagalan, layanan "pengiriman" tetap berfungsi dan dapat menangani pesanan yang sudah dibayar.
 - **Contoh Keberagaman Teknologi:**
Layanan "rekomendasi produk" dapat menggunakan Python dengan machine learning, sementara layanan "pembayaran" menggunakan Java dengan Spring Boot.
-

Aktivitas Pembelajaran:

1. **Diskusi Kelas:**
 - Diskusikan prinsip-prinsip microservices dan bagaimana prinsip ini diterapkan dalam sistem nyata (misalnya: Netflix, Amazon, atau Uber).
 - Identifikasi karakteristik microservices yang paling relevan untuk aplikasi tertentu.
 2. **Studi Kasus:**
 - Berikan contoh aplikasi monolitik dan ajak mahasiswa memikirkan bagaimana aplikasi tersebut dapat dipecah menjadi microservices berdasarkan prinsip dan karakteristik yang telah dipelajari.
 3. **Tugas Kecil:**
 - Minta mahasiswa mencari contoh perusahaan atau aplikasi yang menggunakan microservices dan menjelaskan prinsip serta karakteristik yang diterapkan.
-

Referensi:

1. **Buku:**
 - "Building Microservices" by Sam Newman.
 - "Microservices Patterns" by Chris Richardson.
2. **Artikel:**
 - "Microservices: Principles and Characteristics" oleh Martin Fowler.
 - "What are Microservices?" di DZone.

Topik 3: Desain Arsitektur Microservices

Tujuan Pembelajaran:

1. Memahami konsep Domain-Driven Design (DDD) dan penerapannya dalam microservices.
 2. Mampu mengidentifikasi Bounded Context dan Context Mapping dalam sistem.
 3. Mampu merancang arsitektur microservices berdasarkan prinsip DDD.
-

Materi Pembelajaran:

1. Domain-Driven Design (DDD)

Domain-Driven Design (DDD) adalah pendekatan desain perangkat lunak yang berfokus pada pemahaman mendalam tentang domain bisnis dan menerapkannya ke dalam desain sistem. DDD sangat relevan untuk microservices karena membantu memecah sistem menjadi layanan yang lebih kecil dan terfokus.

- **Domain:**
Area pengetahuan atau aktivitas bisnis yang menjadi fokus sistem (misalnya: e-commerce, perbankan, kesehatan).
- **Subdomain:**
Bagian dari domain yang memiliki fungsionalitas spesifik (misalnya: manajemen produk, pembayaran, pengiriman).
- **Bounded Context:**
Batasan yang jelas di mana model domain tertentu diterapkan. Setiap Bounded Context dapat menjadi layanan microservice.

2. Bounded Context dan Context Mapping

- **Bounded Context:**
Setiap layanan microservice memiliki Bounded Context sendiri, yang mencakup model domain, aturan bisnis, dan data yang relevan.
Contoh:
 - Layanan "Produk" memiliki Bounded Context untuk manajemen katalog produk.
 - Layanan "Pembayaran" memiliki Bounded Context untuk proses transaksi keuangan.
- **Context Mapping:**
Teknik untuk memetakan hubungan antar Bounded Context. Beberapa pola Context Mapping yang umum:
 - **Partnership:** Dua Bounded Context saling bergantung dan berkolaborasi.
 - **Shared Kernel:** Dua Bounded Context berbagi sebagian model domain.
 - **Customer-Supplier:** Satu Bounded Context (Supplier) menyediakan layanan untuk Bounded Context lain (Customer).
 - **Conformist:** Satu Bounded Context mengikuti model domain dari Bounded Context lain.

- **Anti-Corruption Layer:** Lapisan yang mengisolasi Bounded Context dari model domain yang tidak kompatibel.

3. Penerapan DDD dalam Microservices

- **Identifikasi Subdomain:**
Pecah domain bisnis menjadi subdomain yang lebih kecil (misalnya: manajemen produk, pembayaran, pengiriman).
- **Tentukan Bounded Context:**
Setiap subdomain menjadi Bounded Context yang dapat diimplementasikan sebagai layanan microservice.
- **Desain Context Mapping:**
Tentukan hubungan antar Bounded Context menggunakan pola Context Mapping yang sesuai.

4. Contoh Penerapan

- **Aplikasi E-commerce:**
 - **Subdomain:** Manajemen Produk, Pembayaran, Pengiriman, Pengguna.
 - **Bounded Context:**
 - Layanan "Produk" untuk manajemen katalog produk.
 - Layanan "Pembayaran" untuk proses transaksi keuangan.
 - Layanan "Pengiriman" untuk manajemen pengiriman barang.
 - **Context Mapping:**
 - Layanan "Pembayaran" dan "Pengiriman" memiliki hubungan **Customer-Supplier**.
 - Layanan "Produk" dan "Pengiriman" menggunakan **Anti-Corruption Layer** untuk menghindari ketergantungan langsung.

Aktivitas Pembelajaran:

1. **Diskusi Kelas:**
 - Diskusikan contoh aplikasi bisnis (misalnya: e-commerce, perbankan) dan identifikasi subdomain serta Bounded Context-nya.
 - Tentukan pola Context Mapping yang sesuai untuk hubungan antar Bounded Context.
2. **Workshop:**
 - Minta mahasiswa merancang arsitektur microservices untuk aplikasi sederhana (misalnya: sistem perpustakaan atau toko online) menggunakan prinsip DDD.
3. **Tugas Kecil:**

- Berikan studi kasus nyata (misalnya: Netflix, Amazon) dan minta mahasiswa menganalisis Bounded Context dan Context Mapping yang mungkin digunakan.

Referensi:

1. Buku:

- "Domain-Driven Design: Tackling Complexity in the Heart of Software" by Eric Evans.
- "Implementing Domain-Driven Design" by Vaughn Vernon.

2. Artikel:

- "Domain-Driven Design and Microservices" oleh Martin Fowler.
- "Bounded Context and Context Mapping in DDD" di DZone.

Topik 4: Komunikasi Antar Microservices

Tujuan Pembelajaran:

1. Memahami jenis-jenis komunikasi antar microservices (synchronous dan asynchronous).
2. Mampu mengimplementasikan komunikasi synchronous menggunakan REST API dan gRPC.
3. Mampu mengimplementasikan komunikasi asynchronous menggunakan message queue (RabbitMQ, Kafka).

Materi Pembelajaran:

1. Jenis-Jenis Komunikasi Antar Microservices

Komunikasi antar microservices dapat dilakukan secara **synchronous** (langsung) atau **asynchronous** (tidak langsung).

- **Synchronous Communication:**

Komunikasi langsung antara dua layanan, di mana layanan pemanggil (client) menunggu respons dari layanan yang dipanggil (server) sebelum melanjutkan proses.

Contoh: REST API, gRPC.

- **Asynchronous Communication:**

Komunikasi tidak langsung, di mana layanan pemanggil tidak menunggu respons langsung. Pesan dikirim melalui perantara (message broker) dan diproses oleh layanan penerima secara terpisah.

Contoh: RabbitMQ, Apache Kafka.

2. Komunikasi Synchronous

- **REST API:**

- Protokol komunikasi berbasis HTTP.
- Menggunakan metode HTTP seperti GET, POST, PUT, DELETE.

- Contoh: Layanan "Produk" memanggil layanan "Pembayaran" untuk memverifikasi pembayaran.
- Kelebihan: Mudah diimplementasikan, didukung oleh banyak bahasa pemrograman.
- Kekurangan: Tidak cocok untuk komunikasi real-time atau high-throughput.
- **gRPC:**
 - Protokol komunikasi berbasis RPC (Remote Procedure Call) yang dikembangkan oleh Google.
 - Menggunakan protokol HTTP/2 dan format data biner (Protobuf).
 - Contoh: Layanan "Pengiriman" memanggil layanan "Pembayaran" untuk mendapatkan detail transaksi.
 - Kelebihan: Lebih cepat dan efisien dibanding REST API.
 - Kekurangan: Lebih kompleks untuk diimplementasikan.

3. Komunikasi Asynchronous

- **Message Queue:**
 - Sistem yang memungkinkan pengiriman pesan antar layanan melalui antrian (queue).
 - Contoh: RabbitMQ, Apache Kafka.
- **RabbitMQ:**
 - Message broker yang menggunakan protokol AMQP (Advanced Message Queuing Protocol).
 - Contoh: Layanan "Pembayaran" mengirim pesan ke RabbitMQ untuk memberi tahu layanan "Pengiriman" bahwa pembayaran berhasil.
- **Apache Kafka:**
 - Platform streaming data yang dirancang untuk high-throughput dan skalabilitas.
 - Contoh: Layanan "Logging" menerima log dari berbagai layanan melalui Kafka untuk diproses dan disimpan.
- **Pola Asynchronous:**
 - **Event-Driven Architecture:** Layanan bereaksi terhadap event yang diterima (misalnya: pesan pembayaran berhasil).
 - **Pub/Sub (Publish/Subscribe):** Layanan mempublikasikan pesan ke topic, dan layanan lain berlangganan topic tersebut.

4. Contoh Penerapan

- **Komunikasi Synchronous:**

- Layanan "Produk" memanggil layanan "Pembayaran" menggunakan REST API untuk memverifikasi pembayaran.
 - Layanan "Pengiriman" memanggil layanan "Pembayaran" menggunakan gRPC untuk mendapatkan detail transaksi.
 - **Komunikasi Asynchronous:**
 - Layanan "Pembayaran" mengirim pesan ke RabbitMQ untuk memberi tahu layanan "Pengiriman" bahwa pembayaran berhasil.
 - Layanan "Logging" menerima log dari berbagai layanan melalui Kafka untuk diproses dan disimpan.
-

Aktivitas Pembelajaran:

1. Praktikum:

- Implementasi komunikasi synchronous menggunakan REST API:
 - Buat dua layanan (misalnya: layanan "Produk" dan layanan "Pembayaran").
 - Layanan "Produk" memanggil layanan "Pembayaran" untuk memverifikasi pembayaran.
- Implementasi komunikasi asynchronous menggunakan RabbitMQ:
 - Buat dua layanan (misalnya: layanan "Pembayaran" dan layanan "Pengiriman").
 - Layanan "Pembayaran" mengirim pesan ke RabbitMQ, dan layanan "Pengiriman" menerima pesan tersebut.

2. Diskusi Kelas:

- Diskusikan kelebihan dan kekurangan komunikasi synchronous vs asynchronous.
- Identifikasi skenario di mana komunikasi asynchronous lebih cocok digunakan.

3. Tugas Kecil:

- Minta mahasiswa mencari contoh nyata penggunaan komunikasi synchronous dan asynchronous dalam sistem microservices (misalnya: Netflix, Uber).
-

Referensi:

1. Buku:

- "Building Microservices" by Sam Newman.
- "Designing Data-Intensive Applications" by Martin Kleppmann.

2. Artikel:

- "Synchronous vs Asynchronous Communication in Microservices" oleh DZone.

- "Event-Driven Architecture and Microservices" oleh Martin Fowler.

Topik 5: API Gateway dan Service Discovery

Tujuan Pembelajaran:

1. Memahami peran dan fungsi API Gateway dalam arsitektur microservices.
 2. Memahami konsep Service Discovery dan manfaatnya dalam sistem microservices.
 3. Mampu mengimplementasikan API Gateway dan Service Discovery menggunakan tools seperti Spring Cloud Gateway, Kong, Eureka, atau Consul.
-

Materi Pembelajaran:

1. API Gateway

API Gateway adalah komponen penting dalam arsitektur microservices yang bertindak sebagai pintu masuk tunggal (single entry point) untuk semua permintaan dari klien ke layanan microservices.

- **Fungsi API Gateway:**

1. **Routing:** Mengarahkan permintaan ke layanan yang sesuai berdasarkan endpoint.
2. **Load Balancing:** Mendistribusikan beban permintaan ke beberapa instance layanan.
3. **Authentication dan Authorization:** Memvalidasi token dan mengontrol akses ke layanan.
4. **Rate Limiting:** Membatasi jumlah permintaan yang dapat dilakukan oleh klien dalam periode tertentu.
5. **Caching:** Menyimpan respons sementara untuk mengurangi beban pada layanan.
6. **Logging dan Monitoring:** Mencatat log dan metrik untuk keperluan analisis dan pemantauan.

- **Contoh Tools:**

- **Spring Cloud Gateway:** API Gateway berbasis Spring Boot.
- **Kong:** API Gateway open-source yang extensible.
- **Nginx:** Web server yang dapat digunakan sebagai API Gateway.

- **Contoh Penerapan:**

- Klien mengirim permintaan ke API Gateway, yang kemudian mengarahkan permintaan ke layanan "Produk" atau "Pembayaran" berdasarkan endpoint.

2. Service Discovery

Service Discovery adalah mekanisme yang memungkinkan layanan microservices untuk menemukan dan berkomunikasi dengan layanan lain secara dinamis.

- **Fungsi Service Discovery:**

1. **Registrasi Layanan:** Layanan mendaftarkan diri ke Service Registry saat startup.

2. **Penemuan Layanan:** Layanan lain dapat menemukan lokasi (IP dan port) layanan yang dibutuhkan melalui Service Registry.
 3. **Load Balancing:** Mendistribusikan permintaan ke beberapa instance layanan yang tersedia.
- **Pola Service Discovery:**
 - **Client-Side Discovery:** Klien (layanan pemanggil) bertanggung jawab untuk menemukan lokasi layanan yang dibutuhkan.
 - **Server-Side Discovery:** API Gateway atau load balancer bertanggung jawab untuk menemukan lokasi layanan.
 - **Contoh Tools:**
 - **Eureka:** Service Registry yang dikembangkan oleh Netflix.
 - **Consul:** Tool yang menyediakan Service Discovery, health checking, dan konfigurasi terdistribusi.
 - **Zookeeper:** Sistem terdistribusi untuk koordinasi layanan.
 - **Contoh Penerapan:**
 - Layanan "Produk" mendaftarkan diri ke Eureka saat startup.
 - Layanan "Pembayaran" menemukan lokasi layanan "Produk" melalui Eureka untuk melakukan komunikasi.

3. Integrasi API Gateway dan Service Discovery

- API Gateway dapat menggunakan Service Discovery untuk secara dinamis mengarahkan permintaan ke instance layanan yang tersedia.
- Contoh:
 - Klien mengirim permintaan ke API Gateway.
 - API Gateway menggunakan Eureka untuk menemukan lokasi layanan "Produk".
 - API Gateway mengarahkan permintaan ke instance layanan "Produk" yang tersedia.

Aktivitas Pembelajaran:

1. **Praktikum:**
 - Implementasi API Gateway menggunakan Spring Cloud Gateway:
 - Buat API Gateway yang mengarahkan permintaan ke layanan "Produk" dan "Pembayaran".
 - Implementasi Service Discovery menggunakan Eureka:
 - Buat Service Registry dengan Eureka.
 - Daftarkan layanan "Produk" dan "Pembayaran" ke Eureka.

- Gunakan Eureka untuk menemukan lokasi layanan.

2. Diskusi Kelas:

- Diskusikan manfaat dan tantangan penggunaan API Gateway dan Service Discovery.
- Identifikasi skenario di mana API Gateway dan Service Discovery diperlukan.

3. Tugas Kecil:

- Minta mahasiswa mencari contoh nyata penggunaan API Gateway dan Service Discovery dalam sistem microservices (misalnya: Netflix, Amazon).

Referensi:

1. Buku:

- "Building Microservices" by Sam Newman.
- "Microservices Patterns" by Chris Richardson.

2. Artikel:

- "API Gateway in Microservices Architecture" oleh DZone.
- "Service Discovery in Microservices" oleh Martin Fowler.

Topik 7: Keamanan dalam Microservices

Tujuan Pembelajaran:

1. Memahami tantangan keamanan dalam arsitektur microservices.
 2. Mampu mengimplementasikan mekanisme autentikasi dan otorisasi menggunakan OAuth2 dan JWT.
 3. Memahami praktik terbaik untuk mengamankan komunikasi antar microservices.
-

Materi Pembelajaran:

1. Tantangan Keamanan dalam Microservices

Arsitektur microservices memiliki tantangan keamanan yang unik karena sistem terdiri dari banyak layanan yang terdistribusi. Beberapa tantangan tersebut meliputi:

- **Komunikasi Antar Layanan:**
Data yang dikirim antar layanan harus diamankan untuk mencegah penyadapan atau manipulasi.
- **Autentikasi dan Otorisasi:**
Setiap layanan harus memastikan bahwa hanya pengguna atau layanan yang sah yang dapat mengaksesnya.
- **Manajemen Kunci dan Token:**
Kunci enkripsi dan token harus dikelola dengan aman untuk mencegah penyalahgunaan.
- **Keamanan API:**
API yang terpapar ke publik harus dilindungi dari serangan seperti DDoS, SQL injection, atau XSS.

2. Autentikasi dan Otorisasi

- **OAuth2:**
Protokol standar industri untuk otorisasi yang memungkinkan aplikasi untuk mengakses sumber daya atas nama pengguna tanpa membagikan kredensial.
 - **Flow OAuth2:**
 1. Authorization Code Flow: Digunakan untuk aplikasi web.
 2. Implicit Flow: Digunakan untuk aplikasi single-page (SPA).
 3. Client Credentials Flow: Digunakan untuk komunikasi antar layanan.
 - **Contoh Penerapan:**
Layanan "Pembayaran" menggunakan OAuth2 untuk memverifikasi bahwa permintaan berasal dari layanan "Produk" yang sah.
- **JWT (JSON Web Token):**
Token yang digunakan untuk menyimpan informasi pengguna secara terenkripsi.
 - **Struktur JWT:**

1. Header: Menentukan algoritma enkripsi.
2. Payload: Berisi klaim (informasi pengguna).
3. Signature: Digunakan untuk memverifikasi keaslian token.

- **Contoh Penerapan:**

Setelah pengguna login, server mengirim JWT ke klien. Klien menggunakan JWT untuk mengakses layanan lain.

3. Praktik Terbaik Keamanan Microservices

- **Enkripsi Komunikasi:**

Gunakan HTTPS (TLS/SSL) untuk mengamankan komunikasi antar layanan.

- **Validasi Input:**

Validasi semua input untuk mencegah serangan seperti SQL injection atau XSS.

- **Rate Limiting:**

Batasi jumlah permintaan yang dapat dilakukan oleh klien untuk mencegah serangan DDoS.

- **Pemisahan Tanggung Jawab:**

Setiap layanan harus memiliki tanggung jawab keamanan sendiri (misalnya: autentikasi, otorisasi, enkripsi).

- **Audit dan Monitoring:**

Pantau aktivitas sistem dan lakukan audit keamanan secara berkala.

4. Contoh Penerapan

- **Autentikasi dengan OAuth2 dan JWT:**

- Pengguna login melalui layanan "Auth" dan menerima JWT.
- JWT digunakan untuk mengakses layanan "Produk" dan "Pembayaran".

- **Enkripsi Komunikasi:**

- Layanan "Produk" dan "Pembayaran" berkomunikasi menggunakan HTTPS.

- **Rate Limiting:**

- API Gateway membatasi jumlah permintaan yang dapat dilakukan oleh klien dalam satu menit.

Aktivitas Pembelajaran:

1. Praktikum:

- Implementasi autentikasi menggunakan OAuth2 dan JWT:
 - Buat layanan "Auth" yang mengeluarkan JWT setelah pengguna login.
 - Gunakan JWT untuk mengakses layanan "Produk".
- Implementasi HTTPS:

- Konfigurasi layanan untuk menggunakan HTTPS.

2. Diskusi Kelas:

- Diskusikan tantangan keamanan dalam microservices dan bagaimana mengatasinya.
- Identifikasi skenario di mana OAuth2 dan JWT dapat digunakan.

3. Tugas Kecil:

- Minta mahasiswa mencari contoh nyata penerapan keamanan dalam sistem microservices (misalnya: Google, Facebook).

Referensi:

1. Buku:

- "Microservices Security in Action" by Prabath Siriwardena and Nuwan Dias.
- "OAuth 2 in Action" by Justin Richer and Antonio Sanso.

2. Artikel:

- "Securing Microservices with OAuth2 and JWT" oleh DZone.
- "Best Practices for Microservices Security" oleh Martin Fowler.

Topik 9: Containerization dengan Docker

Tujuan Pembelajaran:

1. Memahami konsep containerization dan manfaatnya dalam pengembangan microservices.
 2. Mampu membuat dan mengelola container menggunakan Docker.
 3. Mampu mengimplementasikan Docker untuk menjalankan microservices.
-

Materi Pembelajaran:

1. Konsep Containerization

- **Definisi:**
Containerization adalah teknik untuk mengemas aplikasi beserta semua dependensinya (library, environment, dll.) ke dalam sebuah container yang dapat dijalankan di berbagai lingkungan.
- **Perbedaan dengan Virtual Machine (VM):**
 - Container lebih ringan karena berbagi kernel sistem operasi host.
 - VM membutuhkan sistem operasi lengkap untuk setiap instance.
- **Manfaat Containerization:**
 - Portabilitas: Aplikasi dapat dijalankan di mana saja (lokal, cloud, dll.).
 - Konsistensi: Memastikan lingkungan pengembangan dan produksi sama.
 - Efisiensi: Mengurangi overhead resource dibanding VM.

2. Docker: Tool Containerization Populer

- **Komponen Docker:**
 1. **Docker Image:** Template read-only untuk membuat container.
 2. **Docker Container:** Instance dari image yang sedang berjalan.
 3. **Dockerfile:** File teks yang berisi instruksi untuk membuat image.
 4. **Docker Hub:** Repository publik untuk menyimpan dan berbagi image.
- **Perintah Dasar Docker:**
 - `docker build`: Membuat image dari Dockerfile.
 - `docker run`: Menjalankan container dari image.
 - `docker ps`: Menampilkan container yang sedang berjalan.
 - `docker stop`: Menghentikan container.
 - `docker push`: Mengunggah image ke Docker Hub.
 - `docker pull`: Mengunduh image dari Docker Hub.

3. Menggunakan Docker untuk Microservices

- **Langkah-Langkah:**

1. Buat Dockerfile untuk setiap layanan microservices.
2. Build image dari Dockerfile.
3. Jalankan container dari image.
4. Gunakan Docker Compose untuk mengelola multi-container application.

- **Contoh Dockerfile:**

Dockerfile

Copy

Gunakan base image Java

FROM openjdk:11-jre-slim

Set working directory

WORKDIR /app

Copy file JAR ke container

COPY target/microservice-produk.jar /app/microservice-produk.jar

Expose port

EXPOSE 8080

Command untuk menjalankan aplikasi

CMD ["java", "-jar", "microservice-produk.jar"]

4. Docker Compose untuk Multi-Container Application

- **Definisi:**

Docker Compose adalah tool untuk mengelola aplikasi yang terdiri dari beberapa container.

- **Contoh docker-compose.yml:**

yaml

Copy

version: '3'

services:

produk:

image: microservice-produk:latest

ports:

- "8081:8080"

pembayaran:

image: microservice-pembayaran:latest

ports:

- "8082:8080"

- **Perintah Docker Compose:**
 - docker-compose up: Menjalankan semua container.
 - docker-compose down: Menghentikan dan menghapus semua container.

5. Contoh Penerapan

- **Meng-containerize Layanan Microservices:**
 - Buat Dockerfile untuk layanan "Produk" dan "Pembayaran".
 - Build image dan jalankan container menggunakan Docker Compose.
- **Integrasi dengan Service Discovery:**
 - Gunakan Docker Compose untuk menjalankan layanan "Produk", "Pembayaran", dan Eureka (Service Discovery).

Aktivitas Pembelajaran:

1. Praktikum:

- Buat Dockerfile untuk layanan microservices sederhana (misalnya: layanan "Produk").
- Build image dan jalankan container menggunakan Docker.
- Gunakan Docker Compose untuk menjalankan multi-container application.

2. Diskusi Kelas:

- Diskusikan manfaat containerization dalam pengembangan microservices.
- Identifikasi tantangan dalam menggunakan Docker dan cara mengatasinya.

3. Tugas Kecil:

- Minta mahasiswa mencari contoh nyata penggunaan Docker dalam sistem microservices (misalnya: Netflix, Spotify).
-

Referensi:

1. Buku:

- "Docker Deep Dive" by Nigel Poulton.
- "Using Docker" by Adrian Mouat.

2. Artikel:

- "What is Docker and Why is it Important?" oleh Docker Blog.
- "Docker for Microservices" oleh DZone.

Topik 10: Orchestration dengan Kubernetes

Tujuan Pembelajaran:

1. Memahami konsep orchestration dan peran Kubernetes dalam mengelola container.
2. Mampu mengimplementasikan Kubernetes untuk deploy dan manage microservices.
3. Memahami komponen-komponen utama Kubernetes dan fungsinya.

Materi Pembelajaran:

1. Konsep Orchestration

- **Definisi:**
Orchestration adalah proses otomatisasi untuk mengelola, mengatur, dan mengoptimalkan deployment, scaling, dan operasi container dalam lingkungan yang terdistribusi.
- **Manfaat Orchestration:**
 - **Scaling:** Menyesuaikan jumlah instance container berdasarkan beban.
 - **Load Balancing:** Mendistribusikan traffic ke instance container yang tersedia.
 - **Self-Healing:** Secara otomatis mengganti container yang gagal.
 - **Rolling Updates:** Memperbarui aplikasi tanpa downtime.

2. Kubernetes: Platform Orchestration Populer

- **Definisi:**
Kubernetes (K8s) adalah platform open-source untuk mengotomatisasi deployment, scaling, dan manajemen aplikasi containerized.
- **Komponen Utama Kubernetes:**
 1. **Cluster:** Kumpulan node (mesin fisik/virtual) yang menjalankan container.
 2. **Node:** Mesin worker yang menjalankan container.
 3. **Pod:** Unit terkecil dalam Kubernetes, berisi satu atau lebih container.
 4. **Service:** Abstraksi untuk mengekspos aplikasi yang berjalan di Pod.
 5. **Deployment:** Mengelola lifecycle Pod (deploy, update, scaling).
 6. **ConfigMap dan Secret:** Mengelola konfigurasi dan data sensitif.
 7. **Ingress:** Mengatur akses HTTP/HTTPS ke aplikasi.

3. Menggunakan Kubernetes untuk Microservices

- **Langkah-Langkah:**
 1. Buat file YAML untuk Deployment dan Service.

2. Deploy aplikasi ke Kubernetes cluster.
3. Kelola aplikasi (scaling, update, dll.) menggunakan perintah kubectl.

- **Contoh File YAML:**

- **Deployment:**

yaml

Copy

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: microservice-produk
```

```
spec:
```

```
  replicas: 3
```

```
  selector:
```

```
    matchLabels:
```

```
      app: produk
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: produk
```

```
    spec:
```

```
      containers:
```

```
        - name: produk
```

```
          image: microservice-produk:latest
```

```
          ports:
```

```
            - containerPort: 8080
```

- **Service:**

yaml

Copy

```
apiVersion: v1
```

```
kind: Service
```

```
metadata:
```

name: produk-service

spec:

selector:

app: produk

ports:

- protocol: TCP

port: 80

targetPort: 8080

type: LoadBalancer

4. Fitur Kubernetes untuk Microservices

- **Scaling:**
 - Menyesuaikan jumlah Pod berdasarkan beban menggunakan kubectl scale.
- **Self-Healing:**
 - Kubernetes secara otomatis mengganti Pod yang gagal atau tidak responsif.
- **Rolling Updates:**
 - Memperbarui aplikasi tanpa downtime menggunakan kubectl set image.
- **Service Discovery:**
 - Kubernetes menyediakan DNS internal untuk menemukan layanan.

5. Contoh Penerapan

- **Deploy Layanan Microservices:**
 - Buat Deployment dan Service untuk layanan "Produk" dan "Pembayaran".
 - Deploy ke Kubernetes cluster menggunakan kubectl apply.
- **Scaling dan Load Balancing:**
 - Scale layanan "Produk" menjadi 5 instance.
 - Akses layanan melalui Service yang menyediakan load balancing.

Aktivitas Pembelajaran:

1. **Praktikum:**
 - Buat file YAML untuk Deployment dan Service layanan microservices.
 - Deploy aplikasi ke Kubernetes cluster menggunakan Minikube atau Kubernetes lokal.
 - Lakukan scaling, rolling update, dan monitoring aplikasi.

2. Diskusi Kelas:

- Diskusikan manfaat dan tantangan menggunakan Kubernetes untuk microservices.
- Identifikasi skenario di mana Kubernetes diperlukan.

3. Tugas Kecil:

- Minta mahasiswa mencari contoh nyata penggunaan Kubernetes dalam sistem microservices (misalnya: Google, Spotify).

Referensi:

1. Buku:

- "Kubernetes Up and Running" by Kelsey Hightower, Brendan Burns, and Joe Beda.
- "Kubernetes in Action" by Marko Luksa.

2. Artikel:

- "What is Kubernetes?" oleh Kubernetes Blog.
- "Kubernetes for Microservices" oleh DZone.

Topik 11: Monitoring dan Logging

Tujuan Pembelajaran:

1. Memahami pentingnya monitoring dan logging dalam arsitektur microservices.
 2. Mampu mengimplementasikan monitoring menggunakan Prometheus dan Grafana.
 3. Mampu mengimplementasikan centralized logging menggunakan ELK Stack (Elasticsearch, Logstash, Kibana).
-

Materi Pembelajaran:

1. Pentingnya Monitoring dan Logging

- **Monitoring:**
Proses mengumpulkan, menganalisis, dan memvisualisasikan metrik sistem untuk memastikan kesehatan dan performa aplikasi.
- **Logging:**
Proses merekam event atau aktivitas yang terjadi dalam sistem untuk keperluan audit, debugging, dan analisis.
- **Manfaat:**
 - Mendeteksi masalah secara proaktif.
 - Meningkatkan keandalan dan performa sistem.
 - Memudahkan proses debugging dan troubleshooting.

2. Monitoring dengan Prometheus dan Grafana

- **Prometheus:**
 - Tool open-source untuk monitoring dan alerting.
 - Mengumpulkan metrik dari aplikasi dan menyimpannya dalam time-series database.
 - Fitur:
 - Query language (PromQL) untuk analisis metrik.
 - Alerting untuk notifikasi masalah.
- **Grafana:**
 - Tool untuk memvisualisasikan metrik yang dikumpulkan oleh Prometheus.
 - Fitur:
 - Dashboard yang dapat dikustomisasi.
 - Dukungan untuk berbagai sumber data (Prometheus, Elasticsearch, dll.).
- **Contoh Penerapan:**
 - Setup Prometheus untuk mengumpulkan metrik dari layanan microservices.

- Buat dashboard di Grafana untuk memvisualisasikan metrik seperti CPU usage, memory usage, dan request rate.

3. Centralized Logging dengan ELK Stack

- **ELK Stack:**

Kumpulan tool untuk centralized logging yang terdiri dari:

1. **Elasticsearch:** Database untuk menyimpan dan mencari log.
2. **Logstash:** Tool untuk mengumpulkan, memproses, dan mengirim log ke Elasticsearch.
3. **Kibana:** Tool untuk memvisualisasikan dan menganalisis log.

- **Contoh Penerapan:**

- Setup ELK Stack untuk mengumpulkan log dari berbagai layanan microservices.
- Buat dashboard di Kibana untuk menganalisis log seperti error rate, request pattern, dll.

4. Integrasi Monitoring dan Logging dengan Microservices

- **Metrik yang Perlu Dimonitor:**

- CPU usage, memory usage, dan disk usage.
- Request rate, error rate, dan latency.
- Health check dan status layanan.

- **Log yang Perlu Dikumpulkan:**

- Log aplikasi (info, warning, error).
- Log akses (request dan response).
- Log sistem (startup, shutdown, dll.).

5. Contoh Penerapan

- **Monitoring:**

- Setup Prometheus dan Grafana untuk memonitor layanan "Produk" dan "Pembayaran".
- Buat alert untuk notifikasi jika error rate melebihi threshold.

- **Logging:**

- Setup ELK Stack untuk mengumpulkan log dari layanan "Produk" dan "Pembayaran".
- Buat dashboard di Kibana untuk menganalisis error dan request pattern.

Aktivitas Pembelajaran:

1. **Praktikum:**

- Implementasi monitoring menggunakan Prometheus dan Grafana:
 - Setup Prometheus untuk mengumpulkan metrik dari layanan microservices.
 - Buat dashboard di Grafana untuk memvisualisasikan metrik.
- Implementasi centralized logging menggunakan ELK Stack:
 - Setup ELK Stack untuk mengumpulkan log dari layanan microservices.
 - Buat dashboard di Kibana untuk menganalisis log.

2. Diskusi Kelas:

- Diskusikan manfaat dan tantangan monitoring dan logging dalam microservices.
- Identifikasi metrik dan log yang paling penting untuk dimonitor.

3. Tugas Kecil:

- Minta mahasiswa mencari contoh nyata penggunaan monitoring dan logging dalam sistem microservices (misalnya: Netflix, Uber).

Referensi:

1. Buku:

- "Prometheus: Up and Running" by Brian Brazil.
- "Elasticsearch: The Definitive Guide" by Clinton Gormley and Zachary Tong.

2. Artikel:

- "Monitoring Microservices with Prometheus and Grafana" oleh DZone.
- "Centralized Logging with ELK Stack" oleh Elastic Blog.